(continued)

refers to a mandatory feature, "should" refers to a recommended one, and "may" refers to something that's optional.[4] The goal of the IETF recommendations is essentially to distinguish the really important requirements from the "nice-to-haves."

BDD doesn't use the word "should" in this way. By the time a feature or requirement gets to the stage of writing BDD-style acceptance criteria, there isn't much room for optional requirements. If it's in the acceptance criteria, it needs doing. If you discover that one of the acceptance criteria isn't essential, you can safely remove it.

You've now seen what a typical BDD/TDD development process looks like. BDD unit testing is more an approach than a particular toolset, but there are tools and techniques that can make practicing BDD unit testing easier. We'll look at a few of these in the next few sections.

10.4 Tools that make BDD unit testing easier

Many tools and techniques can smooth the transition to BDD-style unit-testing practices and make the unit tests you write more descriptive and easier to maintain:

- By adopting a descriptive coding style, you can practice BDD unit tests with traditional unit-testing tools such as JUnit and NUnit.
- Other unit-testing tools, such as RSpec, NSpec, and Jasmine, put the emphasis on writing low-level executable specifications in the form of unit tests, which encourages a more rigorous, test-first approach.
- More recent BDD unit-testing tools such as Spock and Spec2 allow for more expressive and powerful low-level specifications, including example-driven specifications.

Let's start by looking at what you can do with traditional unit-testing tools.

10.4.1 Practicing BDD with traditional unit-testing tools

It can sometimes be less intimidating for teams starting out with BDD to keep their existing unit-testing tools. In addition, teams often have a large investment in existing unit tests using a traditional unit-testing library, and they're understandably reluctant to migrate to a new toolset. In this section, we'll look at a few simple steps you can take to start your journey down the road of BDD unit testing without changing your current unit-testing toolset.

WRITE TEST METHOD NAMES THAT DESCRIBE THE BEHAVIOR

The first step toward a more BDD-style approach to unit testing can be as simple as giving your tests more meaningful names. Traditionally, many unit tests use the convention

[4] See Network Working Group, RFC 2119, "Key words for use in RFCs to Indicate Requirement Levels," www.ietf.org/rfc/rfc2119.txt.